

WaveX.NET 1.17.5 SDK Manual

Cometa srl

August 4, 2025

Contents

Contents	1
1 Important Information	3
1.1 Intended use	3
1.2 System requirements	3
1.3 Technical service and support	3
2 Introduction	4
3 General Description	5
4 Architecture	6
4.1 The DaqSystem Module	6
4.2 DeviceState	11
4.3 CaptureConfiguration	11
4.4 Version	16
4.5 ExtVersion	17
4.6 SensorConfiguration	17
4.7 InsoleConfiguration	17
4.8 SensorModel	18
4.9 SensorMode	18
4.10 InsoleSide	18
4.11 InsoleSize	19
4.12 AccelerometerFullScale	19
4.13 GyroscopeFullScale	19
4.14 SensorCheckReport	20
4.15 DeviceStateChangedEventArgs	20
4.16 DataAvailableEventArgs	20
4.17 DataAvailableEventPeriod	23
4.18 DeviceMode	23
4.19 DeviceType	24

4.20	DeviceExtendedType	24
4.21	Sensors State	24
4.22	DeviceError	25
4.23	DaqDeviceExceptionType	28
5	DeviceDependentFunctionalities	31
6	State Machine: transitions and availability of Properties and Methods	32

1 Important Information

The WaveX SDK is a software development tool to be used with the WaveX and Wave Plus System. The SDK is intended to be used as a software component in a finished software product for third-party applications. The function of the SDK is to manage data transfer and to facilitate communication between the WaveX and Wave Plus Systems and third-party software applications.

1.1 Intended use

The SDK is designed to work exclusively with the Picoblue System and is not intended to be used in diagnostic or safety-critical applications.

- Please refer to the hardware manuals for additional important information.

1.2 System requirements

Windows Developer System Requirements:

- Windows 10 and above (64 bit)

1.3 Technical service and support

For information and assistance, please visit:

www.cometasystems.com Contact us

Headquarters: Via Gran Sasso,18

20008 Bareggio (MI) Italy

Tel: +39 (0)2 91410582

2 Introduction

The WaveX.NET software library implements all the functionalities required to fully control WaveX and Waveplus devices through a Personal Computer. The communication with the WaveX and Waveplus devices happens by USB cable, following the USB 3.0 or USB 2.0 specifications, by using a kernel mode USB function driver, which is Windows Driver Foundation (WDF) compliant. The WaveX.NET library can be used in C# or C++ Visual Studio projects for .NET platforms with Windows 11/10 (32/64 bit).

Throughout this manual some parts will be framed in light-blue referring to parts of the SDK concerning only WaveX devices and others in orange for WavePlus devices.

3 General Description

WaveX.NET is a multi-threading library developed in C# language which allows:

- The WaveX and Waveplus devices configuration:
 - data acquisition process configuration
 - sensors configuration
 - analog outputs configuration (available for WaveX system)
- Continuous data acquisition and acquired data integrity evaluation
- Monitoring of external Start/Stop triggers
- Impedance check on the EMG electrodes (available for Waveplus system)
- Offset compensation of signals produced by the accelerometers
- Inertial sensors calibration

Being thread-safe, WaveX.NET also realizes a state machine in order to guarantee higher software reliability, avoiding wrong sequences in the access to the library functionalities, since it generates exceptions in case errors occur during the implementation of its own functionalities.

The WaveX.NET includes the following assemblies:

WaveX.dll

WaveX.Common.dll

WaveX.Sys.dll

PicoBlue.DaqSys.dll

CyUSB.dll

Windows.winmd

4 Architecture

Scheme 1 shows the main software modules.

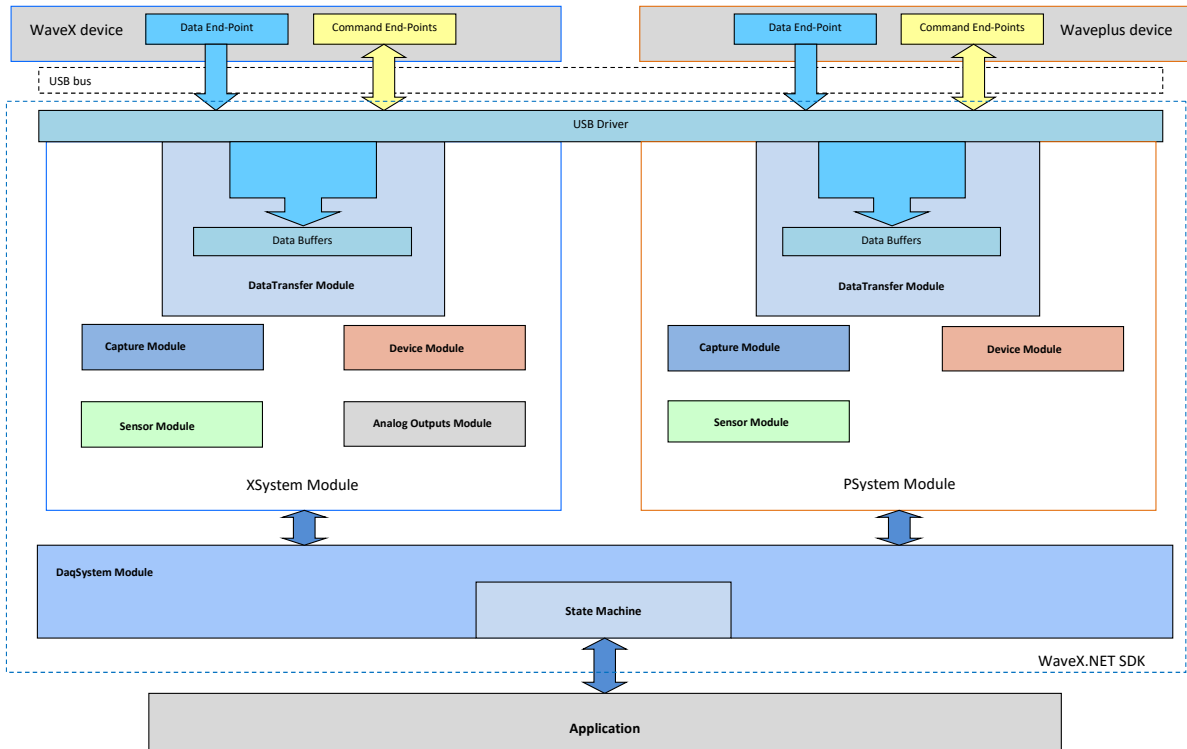


Figure 1: Software modules

4.1 The DaqSystem Module

The `DaqSystem` module (namespace `WaveX`) includes the homonymous class whose methods and properties allow full control of the WaveX and Waveplus devices. This class implements the `IDaqSystem` interface (namespace `WaveX.Interfaces`), which includes the following properties, methods, and event handlers:

WaveX and WavePlus

- `DeviceState` `State { get; }`

Represents the current state of the state machine which regularizes the access to the library (see `DeviceState`)

- `DeviceError` `InitialError { get; }`

Includes the codification of a possible error occurred during the execution of the `DaqSystem` class construction (see `DeviceError`)

- `List<DeviceType> Type { get; }`

Represents the device type list of the WaveX and Waveplus devices connected to USB ports (see [DeviceType](#))

- `List<DeviceExtendedType> ExtendedType { get; }`

Represents the type list of Extended Devices (see [ExtendedDeviceType](#))

- `List<DeviceMode> Mode { get; }`

Represents the device mode list of the WaveX and Waveplus devices connected to USB ports (see [DeviceMode](#))

- `List<int> DeviceInstalledSensors { get; }`

Represents the list of the installed sensors number for each WaveX and Waveplus devices connected to USB ports

- `List<int> ExtendedDeviceInstalledSensors { get; }`

Represents the list of the installed channel numbers for each Extended device type connected

- `List<DeviceDependentFunctionalities> { get; }`

It represents the device-dependent functionalities list of the WaveX and Waveplus devices connected to USB ports (see [DeviceDependentFunctionalities](#))

- `List<IDeviceDependentFunctionalities> ExtendedDeviceDependentFunctionalities { get; }`

It represents the device-dependent functionalities list of the Extended devices connected (see [DeviceDependentFunctionalities](#))

- `void ConfigureCapture(ICaptureConfiguration captureConfiguration);`

Configures the data acquisition process (see [CaptureConfiguration](#))

- `ICaptureConfiguration CaptureConfiguration();`

Gets the current configuration of the data acquisition process (see [CaptureConfiguration](#))

- `SamplingRate CaptureSampleRate { get; }`

Get the sampling rate of the WaveX and WavePlus data. It depends on Acquisition Type protocol and sensors enabled.

- `void StartCapturing(DataAvailableEventPeriod dataAvailableEventPeriod);`

Determines the start of the data acquisition process `dataAvailableEventPeriod` represents the time interval between two consecutive `DataAvailable` events (see [DataAvailable](#) and [DataAvailableEventPeriod](#))

- `void StopCapturing();`

Determines the stop of the data acquisition process

- `void GenerateInternalStartTrigger();`

Generates via software the internal start trigger activation (see [DataAvailableEventArgs](#))

- `void GenerateInternalStopTrigger();`

Generates via software the internal stop trigger activation (see [DataAvailableEventArgs](#))

- `List<IVersion> FirmwareVersion { get; }`

Represents the firmware version list of the WaveX and Waveplus devices connected to the USB ports (see [Version](#))

- `List<IVersion> HardwareVersion { get; }`

Represents the hardware version list of the WaveX and Waveplus devices connected to the USB ports (see [Version](#))

- `IExtVersion SoftwareVersion { get; }`

Represents the Waveplus.Daq.NET software library version (see [ExtVersion](#))

- `int InstalledSensors { get; }`

Represents the total number of the installed sensors

- `int InstalledFootSwSensors { get; }`

Represents the total number of the installed Foot Switch sensors

- `SensorModel[] SensorsModel();`

Returns a vector of `InstalledSensors` elements each of them represents the corresponding sensor model (see [SensorModel](#))

- `void EnableSensor(int sensor);`

Enables the specified sensor when sensor assumes the value between 1 and `InstalledSensors`. If sensor assumes the value 0, all the installed sensors are enabled.

- `void DisableSensor(int sensor);`

Disable (stand-by mode) the selected sensor when sensor assumes the value between 1 and Installedsensors. If sensor assumes the value 0, all sensors are disabled.

- `void EnableInsole(int insole);`

Enables the specified insole when the sensor assumes a value between 1 and InstalledInsoles. If insole assumes the value 0, all the installed insoles are enabled.

- `void DisableInsole(int insole);`

Disable (stand-by mode) the selected insole when insole assumes the value between 1 and Installedsinsoles. If insole assumes the value 0, all insole are disabled.

- `void ConfigureSensor(ISensorConfiguration sensorConfiguration, int sensor);`

Configures the selected sensor when sensor assumes the value between 1 and Installedsensors. If sensor assumes the value 0, all sensors are configured (see [SensorConfiguration](#))

- `ISensorConfiguration SensorConfiguration(int sensor);`

Returns the current configuration of the selected sensor from the “sensor” parameter. This parameter can have value between 1 and InstalledSensors (see [SensorConfiguration](#))

- `event EventHandler<DeviceStateChangedEventArgs> StateChanged;`

Represents the handler of the StateChanged event. This is generated every time the state machine, that manages the access to the library, changes state (see [DeviceStateChangedEventArgs](#))

- `event EventHandler<DataAvailableEventArgs> DataAvailable;`

Represents the handler of the DataAvailable event. This is generated during the acquisition process, when new samples are available (see Scheme 2). To optimize the data processing and transfer, the samples are buffered and published around every DataAvailableEventPeriod ms (see [StartCapturing](#) and [DataAvailableEventPeriod](#))

- `void UpdateDisplay();`

Update the WaveX display according to the current device configuration

- `void EnableFootSwSensors();`
Enables all Foot Switch sensors (2 in total)
- `void DisableFootSwSensors();`
Disables (stand-by mode) all Foot Switch sensors (2 in total)
- `void DetectAccelerometerOffset(int sensor);`
Reads and compensates the offset of x, y, z channels of the selected accelerometers (Waveplus EMG sensor accelerometers), when sensor has value between 1 and InstalledSensors. If sensor has value 0, than all accelerometers are compensated for offset
- `SensorCheckReport[] CheckElectrodeImpedance(int sensor);`
Executes the impedance check on the sensor selected, when sensor has value between 1 and InstalledSensors. If sensor has value 0, than all sensors are checked for impedance. It returns a vector whose elements are of the type `SensorCheckReport`; it includes all check results for all installed sensors (see [SensorCheckReport](#))
- `void CalibrateSensorImuOffset(int sensor);`
Executes the accelerometer and gyroscope offset calibration of the selected inertial sensor, when sensor has value between 1 and InstalledSensors . If sensor has value 0, than it executes the accelerometer and gyroscope offset calibration of all the installed inertial sensors
- `void CalibrateGyroscopeOffset(int sensor);`
Executes the gyroscope offset calibration of the selected inertial sensor, when sensor has value between 1 and InstalledSensors . If sensor has value 0, than it executes the gyroscope offset calibration of all the installed inertial sensors
- `void TurnSensorLedOn(int sensor);`
Activates the blinking of the sensor LED, when sensor has value between 1 and InstalledSensors. If sensor has value 0, than the operation is repeated on all sensors
- `void TurnAllSensorLedsOn();`
Activated the blinking of all sensors, including Footswitch sensors
- `void TurnAllSensorLedsOff();`
Disable the blinking of all sensors, including Footswitch sensors

4.2 DeviceState

Defines the ensemble of all states that the state machine can assume:

```
public enum DeviceState
{
    NotConnected, (No USB device is connected to the USB bus)
    Initializing, (WaveX/Waveplus is initializing)
    CommunicationError, (No communication with WaveX/Waveplus device
        available)
    InitializingError, (WaveX/Waveplus device was not correctly initialized)
    Idle, (WaveX/Waveplus device is ready for use)
    Capturing, (WaveX/Waveplus device is acquiring data)
    ReadingSensorMemory, (WaveX/Waveplus device is reading data from sensor
        memory)
    RemoteRecording (WaveX has started the sensor remote recording)
}
```

4.3 CaptureConfiguration

The class `CaptureConfiguration` implements the interface `ICaptureConfiguration` and includes the following properties:

- `bool ExternalTriggerEnabled { get; set; }`
Enables/disables the external trigger.
 - `int ExternalTriggerActiveLevel { get; set; }`
Defines the level at which the trigger will start/stop acquisition.
 - `bool ExternalTriggerStopDisable { get; set; }`
Disables/Enables the external stop trigger.
 - `int ExternalEventActiveLevel { get; set; }`
Defines the external event level on the RemoteX trigger port.
- `ImuAcqXType IMU_AcqXType { get; set; }`
 - `EmgAcqXType EMG_AcqXType { get; set; }`
 - `EmgImuAcqXType EMG_IMU_AcqXType { get; set; }`
 - `InsoleBtAcqType Insole_BtAcqType { get; set; }`
 - `InsoleRfAcqType Insole_RfAcqType { get; set; }`
 - `InsoleRfProtocol Insole_RfProtocol { get; set; }`
 - `AccelerometerFullScale Insole_AccelerometerFullScale { get; set; }`
 - `GyroscopeFullScale GyroscopeFullScale { get; set; }`
 - `bool[] EnabledInsole { get; set; }`

- `IFootSwTransducerEnabled` `FootSwATransducerEnabled` { get; set; }
- `IFootSwTransducerEnabled` `FootSwBTransducerEnabled` { get; set; }
- `IFootSwTransducerThreshold` `FootSwATransducerThreshold` { get; set; }
- `IFootSwTransducerThreshold` `FootSwBTransducerThreshold` { get; set; }
- `FootSwProtocol` `FootSwProtocol` { get; set; }
- `ImuAcqPType` `IMU_AcqPType` { get; set; }
- `EmgAcqPType` `EMG_AcqPType` { get; set; }

ImuAcqXType

Defines the WaveX inertial sensors acquisition type

```
enum ImuAcqXType
{
    RawAccGyroMagData\_400Hz -- raw data acquisition at 400 Hz (acc/gyro/
        mag)
    RawAccGyroData\_500Hz -- raw data acquisition at 500 Hz (acc/gyro)
    Fused6xData\_400Hz -- 6 axis fused data at 400 Hz
    Fused9xData\_400Hz -- 9 axis fused data at 400 Hz
    Mixed6xData\_200Hz -- fused at 200 Hz + raw (acc/gyro 200Hz, mag 100Hz)
    Mixed9xData\_200Hz -- same as above, 9 axis
}
```

EmgAcqXType

Defines the WaveX EMG and accelerometer sensors acquisition type

```
enum EmgAcqPType
{
    Emg\_2kHz -- EMG acquisition at 2 kHz
}
```

EmgImuAcqXType

Defines the WaveX EMG+IMU sensors acquisition type

```
enum ImuAcqXType
{
    RawAccGyroMagData_284Hz raw data acquisition at 284 Hz
    Fused9xData_142Hz 9 axis fused data (quaternions) acquisition at 142
        Hz
    Fused6xData_284Hz 6 axis fused data (quaternions) acquisition at 284
        Hz
    Fused9xData_71Hz 9 axis fused data (quaternions) acquisition at 71 Hz
    Fused6xData_142Hz 6 axis fused data (quaternions) acquisition at 142
        Hz
    Mixed6xData_142Hz 6 axis fused data (quaternions) acquisition at 142
        Hz and raw data acquisition (accelerometer and gyroscope at 142 Hz,
        magnetometer at 47 Hz)
}
```

ImuAcqPType

Defines the Waveplus inertial sensors acquisition type

```
enum ImuAcqXType
{
    RawAccGyroMagData_284Hz raw data acquisition at 284 Hz
    Fused9xData_142Hz 9 axis fused data (quaternions) acquisition at 142
        Hz
    Fused6xData_284Hz 6 axis fused data (quaternions) acquisition at 284
        Hz
    Fused9xData_71Hz 9 axis fused data (quaternions) acquisition at 71 Hz
    Fused6xData_142Hz 6 axis fused data (quaternions) acquisition at 142
        Hz
    Mixed6xData_142Hz 6 axis fused data (quaternions) acquisition at 142
        Hz and raw data acquisition (accelerometer and gyroscope at 142 Hz,
        magnetometer at 47 Hz)
}
```

EmgAcqPType

Defines the Waveplus EMG and accelerometer sensors acquisition type

```
enum EmgAcqPType
{
    Emg_2kHz_Acc_142Hz EMG acquisition at 2 kHz, accelerometer acquisition
        at 142 Hz
}
```

InsoleRfProtocol

```
enum InsoleRfProtocol
{
    PROPRIETARY_PROTOCOL = 0,
    BT_PROTOCOL = 1,
}
```

InsoleBtAcqType

Defines the insole BT sensors acquisition type

```
enum InsoleBtAcqType
{
    Insole_100Hz = 0,
    Insole_100Hz_RawAccGyroData_500Hz = 1,
    Insole_100Hz_Mixed6xData_200Hz = 2,
    Insole_100Hz_Fused6xData_400Hz = 3,
}
```

```
}
```

InsoleRfAcqType

Defines the insole RF sensors acquisition type

```
enum InsoleRfAcqType
{
    Insole_100Hz = 0,
    InsoleHighRes_100Hz = 1,
    Insole_100Hz_RawAccGyroData_500Hz = 2,
    Insole_100Hz_Fused6xData_400Hz = 3,
    Insole_100Hz_Mixed6xData_200Hz = 4,
}
```

FootSwTransducersEnabled

The class `FootSwTransducersEnabled` implements the interface `IFootSwTransducersEnabled` that includes the following properties:

- `bool T_A { get; set; }` – Enables/Disables the transducer A
- `bool T_1 { get; set; }` – Enables/Disables the transducer 1
- `bool T_5 { get; set; }` – Enables/Disables the transducer 5
- `bool T_T { get; set; }` – Enables/Disables the transducer T

FootSwTransducersThreshold

The class `FootSwTransducersThreshold` implements the interface `IFootSwTransducersThreshold` that includes the following properties:

- `double T_A { get; set; }` – Defines the threshold used for transducer A
- `double T_1 { get; set; }` – Defines the threshold used for transducer 1
- `double T_5 { get; set; }` – Defines the threshold used for transducer 5
- `double T_T { get; set; }` – Defines the threshold used for transducer T

FootSwProtocol

Defines the protocol used to process the Footswitch samples

```
public enum FootSwProtocol
{
    FullFoot,    FullFoot protocol
    HalfFoot,    HalfFoot protocol
    QuarterFoot  QuarterFoot protocol
}
```

4.4 Version

The class `Version` implements the interface `IVersion` that includes the following properties:

- `int Major { get; }`
Represents the Major part of the version number
- `int Middle { get; }`
Represents the Middle part of the version number
- `int Minor { get; }`
Represents the Minor part of the version number

4.5 ExtVersion

The class `ExtVersion` implements the interface `IExtVersion` that includes the following properties:

- `int Major { get; }`
Represents the Major part of the version number
- `int Minor { get; }`
Represents the Minor part of the version number
- `int Build { get; }`
Represents the Build part of the version number
- `int Revision { get; }`
Represents the Revision part of the version number

4.6 SensorConfiguration

The class `SensorConfiguration` implements the interface `ISensorConfiguration` that includes the following properties:

- `SensorModel SensorModel { get; set; }`
Represents the sensor model (see `SensorModel`)
- `SensorMode SensorMode { get; set; }`
Represents the sensor mode (see `SensorMode`)
- `AccelerometerFullScale AccelerometerFullScale { get; set; }`
Represents the full scale value for the x, y, z axes of the accelerometer (see `AccelerometerFullScale`)
- `GyroscopeFullScale GyroscopeFullScale { get; set; }`
Represents the full scale value for the x, y, z axes of the gyroscope (see `GyroscopeFullScale`)

4.7 InsoleConfiguration

The class `InsoleConfiguration` implements the interface `IInsoleConfiguration` that includes the following properties:

- `AccelerometerFullScale AccelerometerFullScale { get; set; }`
Represents the full scale value for the x, y, z axes of the accelerometer (see `AccelerometerFullScale`)
- `GyroscopeFullScale GyroscopeFullScale { get; set; }`
Represents the full scale value for the x, y, z axes of the gyroscope (see `GyroscopeFullScale`)

4.8 SensorModel

Represents the sensor model:

```
public enum SensorModel
{
```

```
    Undefined, (undefined)
    Mini_EmgImu, (Mini EMG + IMU)
    Mini_Emg, (Mini EMG)
    Pico_EmgImu, (Pico EMG + IMU)
    Pico_Emg, (Pico EMG)
    Imu, (Imu)
```

```
    MiniPicoImuWavePlus (Waveplus sensor model)
```

```
}
```

4.9 SensorMode

Represents the sensor mode:

```
public enum SensorMode
{
```

```
    EMG_SENSOR, (EMG)
    INERTIAL_SENSOR, (inertial)
    ANALOG_GP_SENSOR, (general purpose)
```

```
    EMG_INERTIAL_SENSOR, (EMG + inertial)
```

```
    FSW_SENSOR, (foot-switch)
    EMG_ACC_SENSOR (EMG + accelerometer)
```

```
}
```

4.10 InsoleSide

```
public enum InsoleSide
{
```

```
    Undefined = 2,
```

```
    Right = 1,  
    Left = 0  
}
```

4.11 InsoleSize

```
public enum InsoleSize  
{  
    Undefined = 0,  
    Small = 1,  
    Medium = 2,  
    Large = 3,  
    ExtraLarge = 4  
}
```

4.12 AccelerometerFullScale

Represents full scale value for the axis x, y, z of the accelerometer

```
public enum AccelerometerFullScale  
{  
    g_2, (Full scale = 2g)  
    g_4, (Full scale = 4g)  
    g_8, (Full scale = 8g)  
    g_16 (Full scale = 16g)  
}
```

g = gravitational field

4.13 GyroscopeFullScale

Represents full scale value for the axis x, y, z of the gyroscope

```
public enum GyroscopeFullScale  
{  
    dps_250, (Full scale = 250 D/s)  
    dps_500, (Full scale = 500 D/s)  
    dps_1000, (Full scale = 1000 D/s)  
    dps_2000 (Full scale = 2000 D/s)  
}
```

D/s = degree per second

4.14 SensorCheckReport

Defines the result of the impedance check of one sensor

```
public enum SensorCheckReport
{
    NotPassed,
    Passed,
    NotExecuted (the sensor was not enabled or the test was not requested
                for that sensor)
}
```

4.15 DeviceStateChangedEventArgs

The class `DeviceStateChangedEventArgs` implements the following property:

- `DeviceState State { get; }`
Represents the current device state

4.16 DataAvailableEventArgs

The class `DataAvailableEventArgs` implements the following properties:

- `public int ScanNumber { get; set; }`
Represents the number of samples available for each channel
- `public float[,] EmgSamples`
Represents a vector with two dimensions including `ScanNumber` samples for each EMG channel.
The samples are expressed in $[\mu V]$ unit.
The first index identifies the sensor (between 0 and `InstalledSensors - 1`).
The second index identifies the sample (between 0 and `ScanNumber - 1`).
- `public float[,] ImuSamples`
Represents a vector with three dimensions including `ScanNumber` samples for each IMU quaternion component.
The first index identifies the sensor (0 to `InstalledSensors - 1`).
The second index identifies the quaternion component (0 = w, 1 = x, 2 = y, 3 = z).
The third index identifies the sample (0 to `ScanNumber - 1`).
`ImuSamples` samples are available only during IMU fused data acquisition or mixed acquisition.
- `public float[,] AccelerometerSamples`
Represents a vector with three dimensions including `ScanNumber` samples for each Accelerometer channel.
Samples are expressed in $[g]$ unit.

The first index identifies the sensor (0 to InstalledSensors - 1).
The second index identifies the acceleration component (0 = x, 1 = y, 2 = z).
The third index identifies the sample (0 to ScanNumber - 1).

- `public float[,] GyroscopeSamples`
Represents a vector with three dimensions including `ScanNumber` samples for each Gyroscope channel.
Samples are expressed in [D/s] unit.
The first index identifies the sensor (0 to InstalledSensors - 1).
The second index identifies the angular velocity component (0 = x, 1 = y, 2 = z).
The third index identifies the sample (0 to ScanNumber - 1).
- `public float[,] MagnetometerSamples`
Represents a vector with three dimensions including `ScanNumber` samples for each Magnetometer channel.
Samples are expressed in [μ T] unit.
The first index identifies the sensor (0 to InstalledSensors - 1).
The second index identifies the magnetic vector component (0 = x, 1 = y, 2 = z).
The third index identifies the sample (0 to ScanNumber - 1).
- `public short[,] FootSwSamples`
Represents a vector with two dimensions including `ScanNumber` samples for each Footswitch channel.
Samples are expressed in [V] unit.
The first index identifies the sensor (0 to InstalledFSWSensors - 1).
The second index identifies the sample (0 to ScanNumber - 1).
- `public int InsoleScanNumber { get; set; }`
Represents the number of samples available for each insole channel.
- `public int InsoleImuScanNumber { get; set; }`
Represents the number of samples available for each insole IMU channel.
- `public float[,] InsoleSamples`
Represents a vector with three dimensions including `InsoleScanNumber` samples for each EMG channel.
Samples are expressed in [kPa] unit.
The first index identifies the sensor (0 to InstalledInsoles - 1).
The second index identifies the FSR sensor (0 to InsolesSensorNum (64) - 1).
The third index identifies the sample (0 to InsoleScanNumber - 1).
- `public float[,] InsoleImuSamples`
Represents a vector with three dimensions including `InsoleImuScanNumber` samples for each IMU quaternion component.
The first index identifies the sensor (0 to InstalledInsoles - 1).
The second index identifies the quaternion component (0 = w, 1 = x, 2 = y, 3 = z).

The third index identifies the sample (0 to InsoleImuScanNumber - 1).

Samples are available only during IMU fused or mixed acquisition.

- **public float[,] InsoleAccelerometerSamples**
Represents a vector with three dimensions including `InsoleImuScanNumber` samples for each Accelerometer channel.
Samples are expressed in [g] unit.
The first index identifies the sensor (0 to InstalledInsoles - 1).
The second index identifies the acceleration component (0 = x, 1 = y, 2 = z).
The third index identifies the sample (0 to InsoleImuScanNumber - 1).
- **public float[,] InsoleGyroscopeSamples**
Represents a vector with three dimensions including `InsoleImuScanNumber` samples for each Gyroscope channel.
Samples are expressed in [D/s] unit.
The first index identifies the sensor (0 to InstalledInsoles - 1).
The second index identifies the angular velocity component (0 = x, 1 = y, 2 = z).
The third index identifies the sample (0 to InsoleImuScanNumber - 1).
- **public float[] SyncSamples**
Reserved.
- **public short[] SensorStates**
Represents a vector including the state of each sensor.
The index identifies the sensor (0 to InstalledSensors - 1).
(See Sensors State)
- **public short[] InsoleStates**
Represents a vector including the state of each insole sensor.
The index identifies the sensor (0 to InstalledInsoles - 1).
(See Sensors State)
- **public short[] FootSwSensorStates**
Represents a vector including the state of each foot-switch.
The index identifies the sensor (0 to InstalledFSWSensors - 1).
(See Sensors State)
- **public bool StartTriggerDetected**
Indicates if a start trigger signal was detected.
- **public bool StopTriggerDetected**
Indicates if a stop trigger signal was detected.
- **public int StartTriggerScan**
Represents the position (sample index value) in the data vectors where the start trigger was detected. If none detected, value is 0.

- `public int StopTriggerScan`
Represents the position (sample index value) in the data vectors where the stop trigger was detected. If none detected, value is 0
 - `public bool StartInsoleTriggerDetected`
 - `public bool StopInsoleTriggerDetected`
 - `public int StartInsoleTriggerScan`
 - `public int StopInsoleTriggerScan`
 - `public bool StartInsoleImuTriggerDetected`
 - `public bool StopInsoleImuTriggerDetected`
 - `public int StartInsoleImuTriggerScan`
 - `public int StopInsoleImuTriggerScan`
 - `public ushort[] RemoteControlEvents`
 - `public ushort RemoteControlState`

4.17 DataAvailableEventPeriod

Represents the time interval between two consecutive `DataAvailable` events

```
public enum DataAvailableEventPeriod
{
    ms_100, Interval = 100 ms
    ms_50, Interval = 50 ms
    ms_25, Interval = 25 ms
    ms_10 Interval = 10 ms
}
```

Note: interval values less than 100 ms require a very short execution time of the `DataAvailableEvent` event handler.

4.18 DeviceMode

```
public enum DeviceMode
{
    Undefined,
    Master,
    Slave,
    MasterSlave,
}
```

4.19 DeviceType

Defines the type of the connected device.

```
public enum DeviceType
{
    Undefined,
    WavePlus,
    WavePlusLight,
    WaveX,
    PicoBT,
}
```

4.20 DeviceExtendedType

Defines the type of the other (supported) connected devices

```
public enum DeviceExtendedType
{
    Undefined,
    InsoleBT,
    AnalogBoard
}
```

4.21 Sensors State

Sensor state is represented by a 16-bit value formatted according to the following table:

D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
0	0	0	0	0	0	0	0	0	0	0	BLAF	BL3	BL2	BL1	BL0

where BL3 – BL0 represent the sensor's battery charge level bits, and BLAF represents the battery level available flag:

BLAF	BL3	BL2	BL1	BL0	Battery charge level
1	0	0	0	0	0%
1	0	0	0	1	
1	0	0	1	0	
1	0	0	1	1	
1	0	1	0	0	
1	0	1	0	1	
1	0	1	1	0	
1	0	1	1	1	
1	1	0	0	0	
1	1	0	0	1	
1	1	0	1	0	
1	1	0	1	1	
1	1	1	0	0	
1	1	1	0	1	
1	1	1	1	0	
1	1	1	1	1	
0	X	X	X	X	Not Available

Note: For the Waveplus system, the sensor battery charge level information is available only during EMG/Accelerometer, Foot-switch, and IMU raw-data acquisitions.

4.22 DeviceError

Defines the possible errors:

```
public enum DeviceError
{
    Success = 0,
    UnspecifiedError = 1,
    DeviceNotConnected = 2,
    SendingCommand = 3,
    ReceivingCommandReply = 4,
    DeviceErrorExecutingCommand = 5,
    ConfiguringCapture = 6,
    ConfiguringAudio = 7,
    WrongCaptureConfigurationImuAcqType = 8,
    WrongCaptureConfigurationSamplingRate = 9,
    WrongCaptureConfigurationSamplingRateFromDevice = 10,
    WrongCaptureConfigurationDataAvailableEventPeriod = 11,
    WrongAudioConfigurationSpeakerLevel = 12,
    WrongAudioConfigurationSource = 13,
    WrongAudioConfigurationSourceFromDevice = 14,
    WrongSensorConfigurationAccelerometerFullScale = 15,
    WrongSensorConfigurationGyroscopeFullScale = 16,
```

```
WrongSensorConfigurationSensorType = 17,  
WrongFootSwProtocol = 18,  
WrongDeviceTypeFromDevice = 19,  
WrongDeviceMode = 20,  
WrongSensorNumber = 21,  
WrongFootSwSensorNumber = 22,  
FootSwSensorNotInstalled = 23,  
ReadingBackCaptureConfigurationSamplingRate = 24,  
ReadingBackAudioConfigurationAudioSource = 25,  
ReadingBackAudioConfigurationSpeakerVolumeLevel = 26,  
ReadingBackAudioConfigurationSpeakerEnabled = 27,  
ReadingBackCommunicationTestData = 28,  
ReadingBackSensorCommandBuffer = 29,  
ParsingFpgaConfigurationFile = 30,  
SendingFpgaConfigurationImage = 31,  
ReceivingUpdateFpgaConfigurationCommandReply = 32,  
DeviceErrorExecutingUpdateFpgaConfigurationCommand = 33,  
FpgaConfiguredFlagNotAsserted = 34,  
ReadingFpgaConfiguredFlag = 35,  
DataTransferThreadStartingTimeout = 36,  
ActionNotAllowedInTheCurrentDeviceState = 37,  
FeatureNotSupportedByTheDevice = 38,  
ActionNotAllowedInTheCurrentDataTransferState = 39,  
WrongTrialNumber = 40,  
WrongDeviceState = 41,  
WrongDeviceAction = 42,  
TimeoutExecutingSensorCommand = 43,  
CommandNotExecutedByAllTheSensors = 44,  
TimeoutReadingSensorMemoryStatus = 45,  
SensorMemoryHeaderCorrupted = 46,  
SensorMemoryWrongTrialAddressFormat = 47,  
SensorMemoryTrialAddressOutOfRange = 48,  
WrongDaqTimeOutValue = 49,  
BadSensorCommunication = 50,  
SyncBuffer1Overrun = 51,  
SyncBuffer2Overrun = 52,  
ImuCalibrationNotAvailable = 53,  
SaveMPStatusNotAvailable = 54,  
FunctionNotAvailable = 55,  
WrongDeviceRFChannelFromDevice = 56,  
WrongImuAcqType = 57,  
SensorDoesNotProvideSelectiveMemoryReading = 58,  
ConflictingDeviceModeFromEEPROMandFromMicro = 59,  
WrongEmgHwGain = 60,
```

```
WrongCaptureConfigurationImuAcqPType = 61,  
WrongCaptureConfigurationImuAcqXType = 62,  
WrongCaptureConfigurationEmgAcqPType = 63,  
WrongCaptureConfigurationEmgAcqXType = 64,  
WrongCaptureConfigurationEmgImuAcqXType = 65,  
WrongCaptureConfigurationEmgAcqBType = 66,  
WrongCaptureConfigurationImuAcqBType = 67,  
UnavailableBSystemEmgAcqType = 68,  
UnavailablePSystemEmgAcqType = 69,  
UnavailablePSystemImuAcqType = 70,  
SendingAnalogOutputsConfiguration = 71,  
ReceivingAnalogOutputsConfigurationReply = 72,  
DeviceErrorSendingAnalogOutputsConfiguration = 73,  
ReadingBackAnalogOutputsConfiguration = 74,  
SendingSensorMemoryStatusRequest = 75,  
ReceivingSensorMemoryStatusReply = 76,  
ExecutingSensorMemoryStatusRequest = 77,  
DataPacketLost = 78,  
WiFiSensorMemoryReadingError1 = 79,  
WiFiSensorMemoryReadingError2 = 80,  
SendingSensorIdRequest = 81,  
ReceivingSensorIdReply = 82,  
ExecutingSensorIdRequest = 83,  
WrongSensorConfigurationSensorId = 84,  
WrongRFDeviceChannel = 85,  
WrongRFSensorChannel = 86,  
WiFiSensorMemoryReadingError3 = 87,  
WiFiSensorMemoryReadingError4 = 88,  
WiFiSensorMemoryReadingError5 = 89,  
WiFiSensorMemoryReadingError6 = 90,  
WiFiSensorMemoryReadingError7 = 91,  
WiFiSensorMemoryReadingError8 = 92,  
WrongDeviceCurrentTrialIdFromDevice = 93,  
WrongCommandCodeFromSensor = 94,  
WrongRFChannel = 95,  
WrongRFChannelName = 96,  
WrongSensorId = 97,  
WrongSensorIdCode = 98,  
WrongSensorIdName = 99,  
WrongInsoleSize = 100,  
WrongInsoleAcqType = 101,  
WrongHDsEmgAcqType = 102,  
WrongAnalogAcqType = 103,  
WrongInsoleRFProtocol = 104,
```

```
SyncBufferIOverrun = 105,  
SyncBufferIImuOverrun = 106,  
SyncBufferHOverrun = 107,  
SyncBufferAOverrun = 108,  
DataFifoIsFull = 109,  
TimeoutWaitingForDeviceStateChange = 110,  
WrongAnalogTriggerMode = 111,  
WrongVoltageFullScale = 112,  
}
```

4.23 DaqDeviceExceptionType

Defines the possible exception types generated by the library:

```
enum DaqDeviceExceptionType  
{  
    deviceNotConnected = 0,  
    unableToStartCaptureDataTransfer = 1,  
    unableToStartImpedanceDataTransfer = 2,  
    unableToStartCapturing = 3,  
    unableToStopCapturing = 4,  
    unableToReadSensorMemoryStatus = 5,  
    unableToGetCaptureConfiguration = 6,  
    unableToSetCaptureConfiguration = 7,  
    unableToGetInstalledSensors = 8,  
    unableToGetDeviceType = 9,  
    unableToConfigureSensor = 10,  
    unableToGetSensorConfiguration = 11,  
    unableToTurnInternalTrigger_OFF = 12,  
    unableToTurnInternalTrigger_ON = 13,  
    unableToEnableSensor = 14,  
    unableToDisableSensor = 15,  
    unableToEnableFootSwSensor = 16,  
    unableToDisableFootSwSensor = 17,  
    unableToDetectAccelerometerOffset = 18,  
    unableToCheckElectrodeImpedance = 19,  
    unableToGetElectrodeImpedanceReport = 20,  
    unableToTurnSensorLedOn = 21,  
    unableToTurnFootSwSensorLedOn = 22,  
    unableToTurnAllSensorLedsOn = 23,  
    unableToTurnAllSensorLedsOff = 24,  
    unableToStartSensorMemoryRecording = 25,  
    unableToStopSensorMemoryRecording = 26,  
    unableToClearSensorMemory = 27,  
    unableToFormatSensorMemory = 28,  
}
```

```
unableToStartSensorMemoryReading = 29,  
unableToStartSensorSelectiveMemoryReading = 30,  
unableToStopSensorMemoryReading = 31,  
unableToEnableSensorMemoryMode = 32,  
unableToDisableSensorMemoryMode = 33,  
unableToCalibrateSensorImu = 34,  
unableToCalibrateSensorGyroscope = 35,  
unableToSaveMPStatus = 36,  
unableToGetFirmwareVersion = 37,  
unableToGetHardwareVersion = 38,  
unableToSetAudioConfiguration = 39,  
unableToGetAudioConfiguration = 40,  
unableToConvertParameter = 41,  
unableToUpdateFirmware = 42,  
unableToGetFPGAConfigFlag = 43,  
unableToUpdateDeviceBoardEEPROMInfo = 44,  
unableToGetDeviceBoardEEPROMInfo = 45,  
unableToSynchronizeData = 46,  
unableToChangeDeviceRFChannel = 47,  
unableToChangeSensorRFChannel = 48,  
unableToGetDeviceRFChannel = 49,  
unableToSetFirstImuCalibrationStep = 50,  
unableToSetNextImuCalibrationStep = 51,  
unableToSetIMUAcqType = 52,  
unableToGetDeviceDependentFunctAvailability = 53,  
unableToReadSensorInfo = 54,  
unableToEnableRemoteControl = 55,  
unableToDisableRemoteControl = 56,  
unableToCalibrateSensorAccelerometer = 57,  
unableToSetEmgImuAcqType = 58,  
unableToGetDeviceSensorsId = 59,  
unableToSetEmgAcqType = 60,  
unableToSetImuAcqType = 61,  
unableToSetAnalogOutputsConfiguration = 62,  
unableToGetAnalogOutputsConfiguration = 63,  
unableToEnableDeviceWiFi = 64,  
unableToDisableDeviceWiFi = 65,  
unableToEnableSensorWiFi = 66,  
unableToDisableSensorWiFi = 67,  
unableToSetSensorMemoryTrialId = 68,  
unableToStartSensorMemoryDataTransfer = 69,  
unableToFindAndInitializeBDevice = 70,  
unableToDisconnectBDevice = 71,  
unableToSetDeviceDmaBufferSize = 72,
```

```
unableToDisableWiFi = 73,  
unableToGetSensorMemoryStatus = 74,  
unableToReadSensorId = 75,  
unableToSetSensorSleepMode = 76,  
unableToSaveConfigurationOnXDevice = 77,  
unableToGetDeviceCurrentTrialId = 78,  
unableToGetSensorInfo = 79,  
unableToCalibrateSensorEmg = 80,  
unableToGetFixedMemoryId = 81,  
unableToExecuteImuFactoryCalibration = 82,  
  
unableToFindAndInitializeADevice = 83,  
unableToFindAndInitializeMDevice = 84,  
unableToFindAndInitializeSDevice = 85,  
  
unableToGoToSensorSystemOff = 86,  
unableToEnableInsole = 87,  
unableToDisableInsole = 88,  
  
UnableToAccessInsoleDataFiFo = 89,  
UnableToSendCommandToDongle = 90,  
unableToDisconnectIDevice = 91,  
  
unableToFindAndInitializeHDevice = 92,  
unableToDisconnectHDevice = 93,  
  
unableToConfigureADeviceTasks = 94,  
unableToCptureADeviceDigitalData = 95,  
unableToCptureADeviceAnalogData = 96,  
unableToConfigureADeviceDigitalTrigger = 97,  
unableToEmptyADeviceBuffer = 98,  
}
```

5 DeviceDependentFunctionalities

Starting from Waveplus.Daq.Net library (SDK) Rel 4.1.2.0, the user can get device-dependent functionalities availability by reading the new field exposed in [IDaqSystem](#) interface:

```
List<IDeviceDependentFunctionalities > DeviceDependentFunctionalities { get; }
```

It represents the device-dependent functionalities list of the VaveX/Waveplus devices connected to USB ports

The class [DeviceDependentFunctionalities](#) implements the interface [DeviceDependentFunctionalities](#) and includes the following properties:

- [DeviceType](#) DeviceType { get; }
- [DeviceExtendedType](#) DeviceExtendedType { get; }
- [bool](#) RfFrequencySettingSupported { get; }
it assumes the value true if the RF frequency changing is supported by the device, false if the functionality is not supported.
- [bool](#) SelectiveMemoryReadingSupported { get; }
it assumes the value true if the selective memory reading is supported by the device, false if the functionality is not supported.
- [bool](#) GyroscopeOffsetDetectionSupported { get; }
- [bool](#) MPLStatusSavingSupported { get; }
- [bool](#) SensorInfoReadingSupported { get; }
- [bool](#) RemoteStartStopRecordingSupported { get; }
- [bool](#) SensorMemoryReadingSupported { get; }
- [bool](#) ImpedanceCheckingSupported { get; }
- [bool](#) AdvancedSettingSupported { get; }
- [bool](#) AnalogOutputsSupported { get; }
- [bool](#) RemoteRecordingStatusSupported { get; }

The access to these properties is allowed in the following device states:

Idle

Capturing

ReadingSensorMemory

6 State Machine: transitions and availability of Properties and Methods

The access to the library functionalities is regulated by a state machine aimed at improving the software reliability. In the following [Scheme](#) the allowed transitions and the available methods/properties are highlighted for every state.

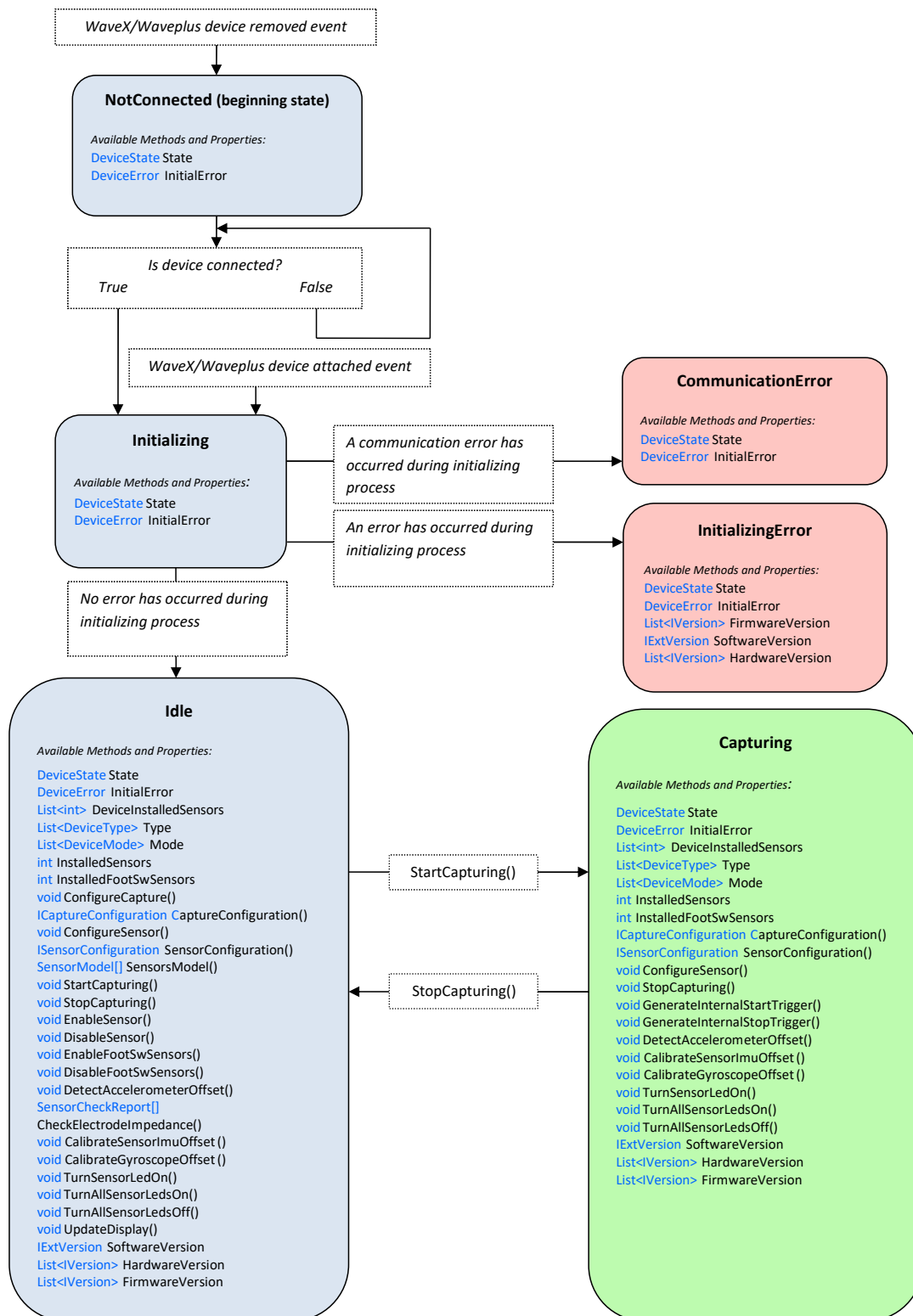


Figure 2: State Machine Scheme